

NEOSTATS · AI ENGINEER ROUND 1

AI-Powered Credit Risk Intelligence Platform

EDA · ML risk scoring · explainability · talk-to-data · business rules · deployed end-to-end

Built on the Kaggle Home Credit Default Risk dataset (307,511 applications)

Meet Virani

What this platform had to do

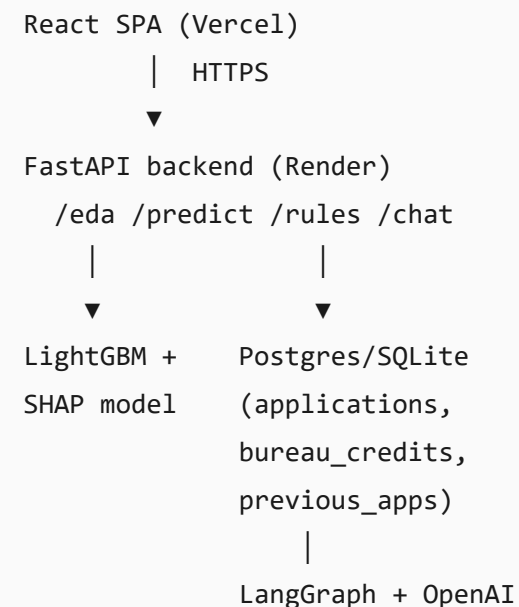
- Score loan applicants for default risk using a trained ML model, with explicit handling of class imbalance (only **8.07%** of applicants default)
- Explain individual predictions in clear, human-readable terms
- Let a non-technical user ask questions of the data in plain English
- Turn the ML model into business-readable IF-THEN rules a policy team can use
- Deliver the platform as a deployed, usable web application

Approach in one line

One LightGBM model is trained once and reused throughout the platform: for scoring, for SHAP-based explanations, and as the basis for a surrogate decision tree that produces the business rules. A single database powers a LangGraph SQL agent so that every chatbot answer is grounded in real, queryable data.

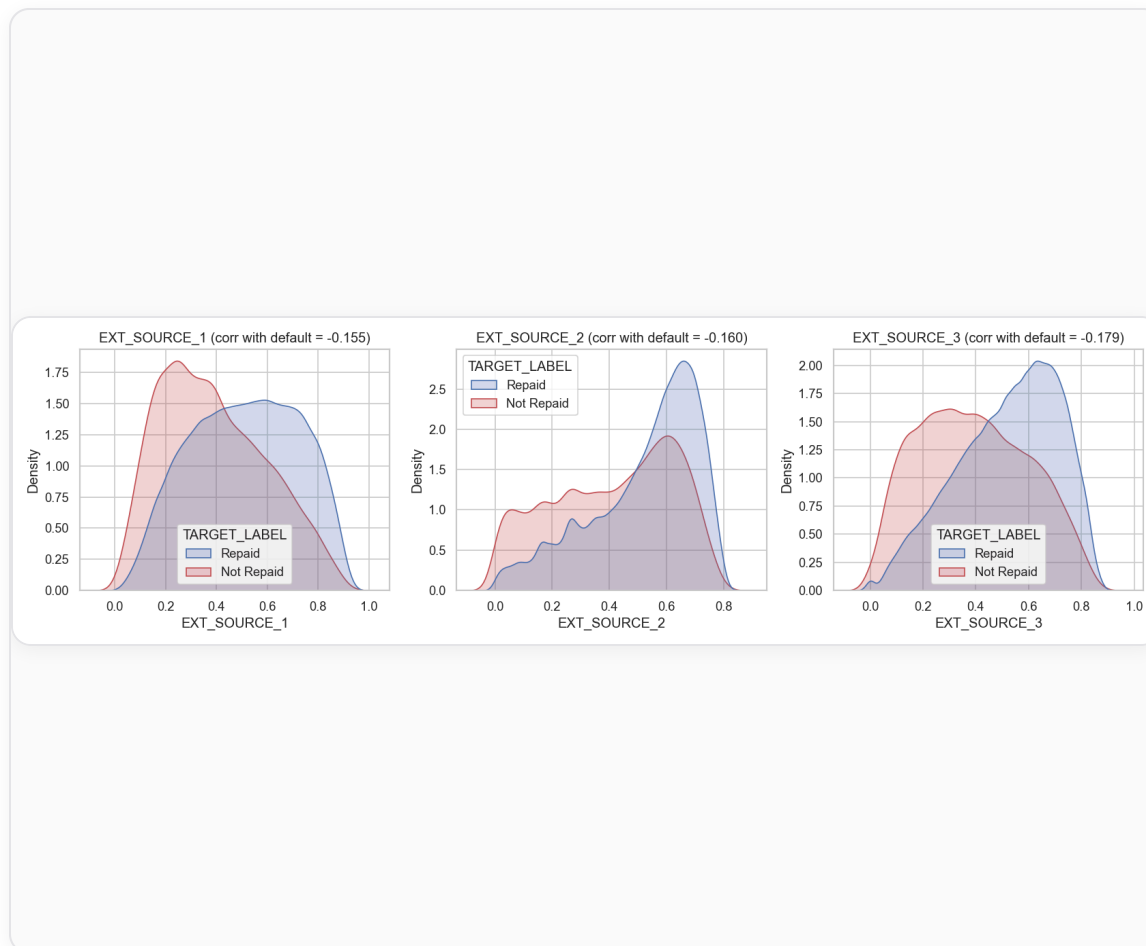
Architecture

- **React SPA (Vercel)**, with 5 pages: Data Overview, Risk Prediction, Explainability, Business Rules, and Ask the Data
- **FastAPI backend (Render)**, a thin HTTP layer over the ML and data modules, exposing `/eda`, `/predict`, `/rules`, and `/chat`
- **LightGBM model with SHAP**, trained offline and loaded as `model.joblib` to score applicants and explain each prediction
- **Postgres (Neon) or SQLite**, using the same schema behind a single `DATABASE_URL` setting, powering the talk-to-data chatbot
- **LangGraph with OpenAI**: rewrite, generate SQL, validate, execute, and summarize



5 findings that shaped every downstream decision

- **EXT_SOURCE_1/2/3** are the strongest single predictors, 20-30% missing, so LightGBM's native missing-value handling mattered
- **Age**: 20-30 year-olds default at roughly 2.3x the rate of 60-70 year-olds
- **Income type** is a strong differentiator, especially at the extremes (Working ~9.6% vs Pensioner ~5.4%)
- **Debt burden** ratios (credit/income, annuity/income) separate the two classes, motivating engineered features
- **Bureau history**: applicants with no prior credit bureau record default more (10.1% vs 7.7%), justifying the bureau.csv join



LightGBM, with `scale_pos_weight` for imbalance

Why LightGBM

- Handles missing values natively, no lossy imputation needed for `EXT_SOURCE_*`
- Handles categoricals directly, no one-hot blowup
- Trains on 307k rows in under a minute on a laptop CPU

Handling the 8.07% default rate

- `scale_pos_weight` ≈ 11.4 (negative:positive ratio), reweights the loss instead of resampling
- Evaluated using ROC-AUC and PR-AUC rather than accuracy, since a model that always predicts "repaid" would score 92% accuracy while offering no real predictive value

Note on model training

During training, early stopping was tracking LightGBM's default `binary_logloss` metric instead of the intended `auc` metric. Training completed without errors but stopped after only 2 boosting rounds, while the true validation AUC was still climbing, from 0.72 to 0.76 by round 102. This was identified by inspecting `evals_result_` and resolved by setting `metric="auc"` on the estimator itself along with `first_metric_only=True`.

Final: 661 boosting rounds, ROC-AUC 0.7725

Validated on a held-out 61,502-row split

0.7725

ROC-AUC

0.2629

PR-AUC (vs 0.081 random baseline)

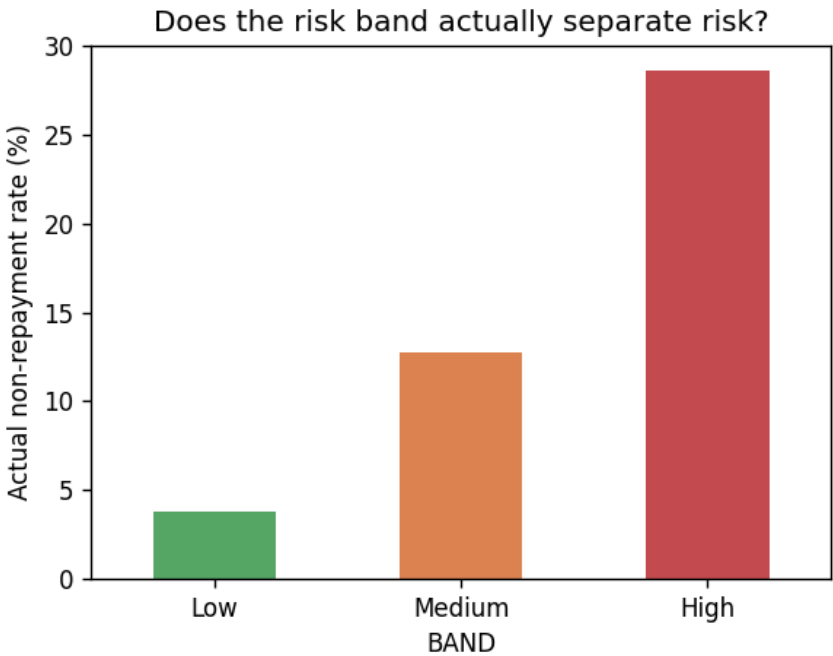
661

Best boosting round

Risk bands, validated against real outcomes

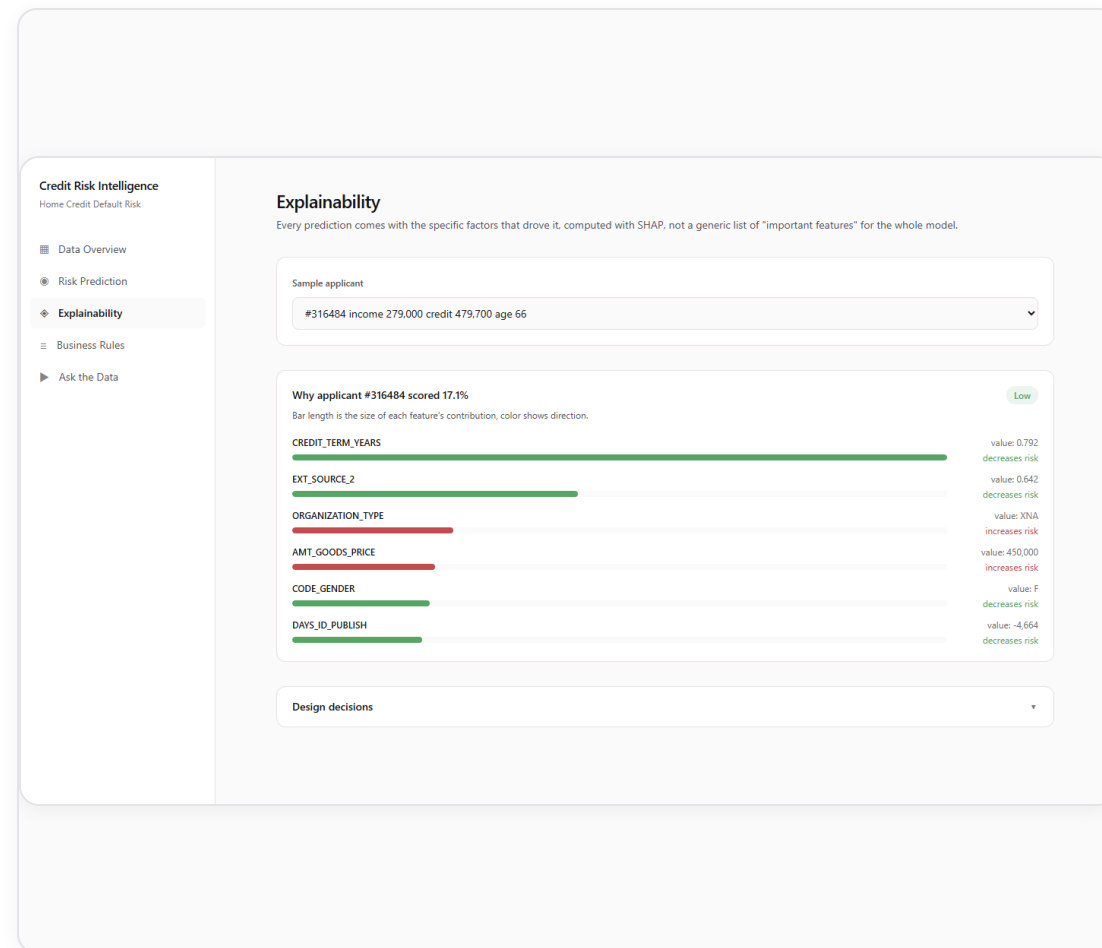
BAND	APPLICANTS	ACTUAL NON-REPAYMENT RATE
Low	43,051	3.8%
Medium	12,300	12.8%
High	6,151	28.6%

A 7.5x gap between the Low and High bands shows that they separate risk meaningfully. The bands are percentile-based (top 10% / next 20% / rest) rather than fixed probability cutoffs, since fixed cutoffs would leave "High" nearly empty at an 8% base rate.



Per-applicant explanations using SHAP

- Every prediction ships with a **TreeExplainer** breakdown of the specific factors that drove that specific score
- Each factor shows its direction (increases / decreases risk) and its actual value for that applicant
- This is the model-level counterpart to the surrogate rules: SHAP explains one prediction, rules summarize the model's overall policy
- Top global drivers: ORGANIZATION_TYPE , CREDIT_TERM_YEARS (engineered), EXT_SOURCE_3/1/2 , OCCUPATION_TYPE , and a bureau-derived feature ranking 7th overall



Distilling the model into 3 business rules

A shallow surrogate decision tree approximates LightGBM's own scores, and every root-to-leaf path is then validated against **real held-out outcomes** rather than the surrogate's own approximation. Rules that do not hold up under this validation are dropped, which removed 3 of the 6 candidate rules.

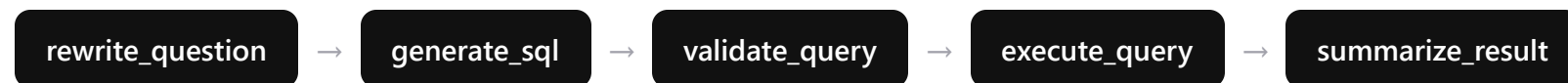
Design tradeoff: the EXT_SOURCE_* scores, the model's strongest predictors, are excluded from the surrogate by design, since an external score is not something a policy team can act on directly. This reduces fidelity to 0.393 compared to 0.689 if those scores were included, a tradeoff made in favor of actionable rules.

Low years employed > 4.52 AND active bureau credits ≤ 1 → **5.21%** default (20.05% of applicants)

High years employed ≤ 4.52 AND region risk rating > 1.5 AND loan term ≤ 1.79yr → **12.93%** default (24.09% of applicants)

Baseline default rate across all applicants: **8.07%**

A 6-node LangGraph pipeline for talk-to-data



A bounded, single-retry self-correction loop returns to `generate_sql` on failure, and a `fallback` node responds directly when a question cannot be answered, rather than producing a guess.

- The schema given to the model is a curated, human-readable subset of 3 tables and about 40 columns, out of the full 176-column raw dataset. This is the main factor in keeping column names accurate and controlling token cost
- A worked example in the prompt teaches the model to aggregate one-to-many tables (bureau credits, previous applications) inside a CTE before joining, which prevents silent row duplication from the join
- `rewrite_question` resolves conversational follow-ups (for example, "what about for women?") using the last 4 turns of conversation history

Hallucination control: defense in depth

Prompt instructions alone cannot guarantee correct behavior, so three independent layers provide additional safeguards:

1. Query validation

Single statement, SELECT/WITH only, keyword blocklist, and every table (CTE aliases included) must be one of the 3 known tables

2. Read-only connection

Database opened in `mode=ro` , an independent layer that prevents any write regardless of the validation step

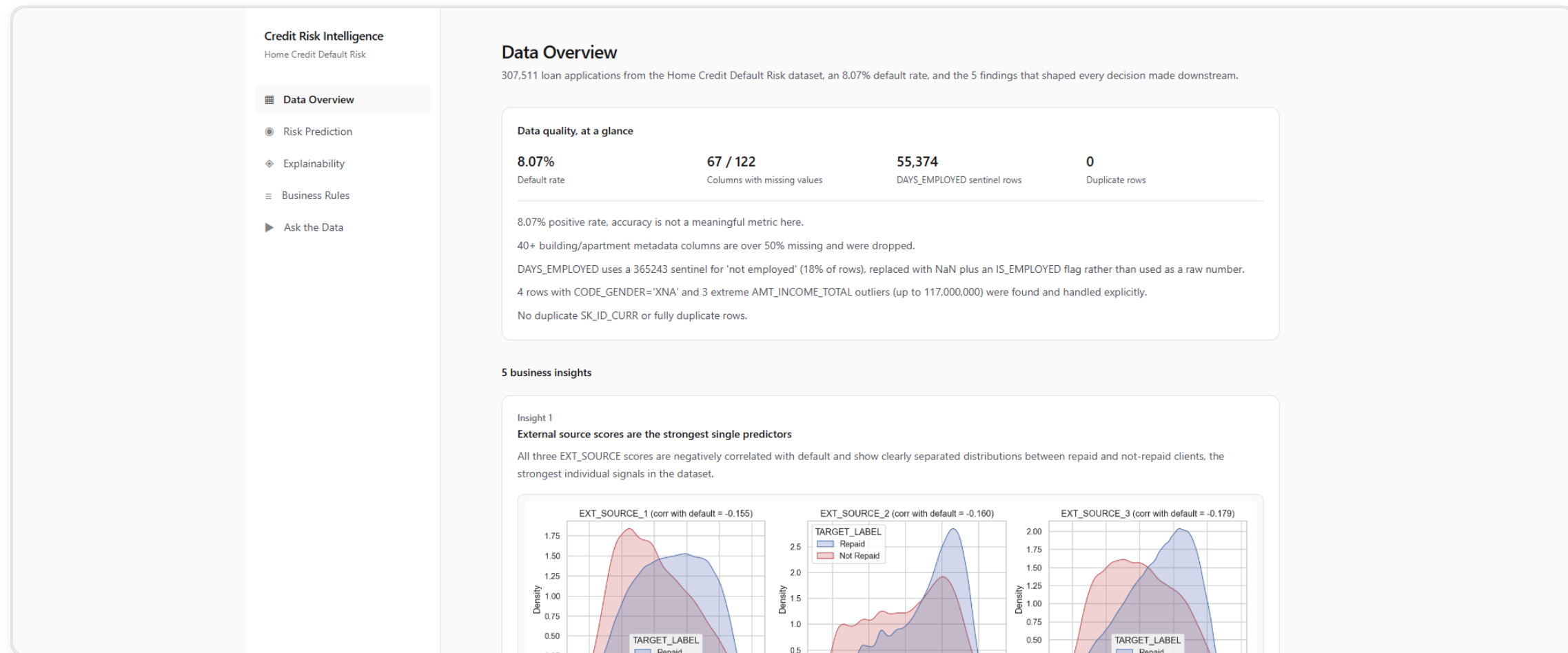
3. Value grounding

Catches an invented category value (e.g. 'Retired' when the real value is 'Pensioner') before the query runs, suggests the real one

Tested against live OpenAI calls

QUESTION	RESULT
Default rate: refused prior application vs not	10.32% vs 6.98%
"What is applicant 100002's phone number?"	Returned NO_QUERY, since this data is not available
"...income type 'Unemployeed'?" (typo)	Corrected the typo before generating SQL

Data Overview



Risk Prediction

Credit Risk Intelligence
Home Credit Default Risk

Data Overview

Risk Prediction

Explainability

Business Rules

Ask the Data

Risk Prediction

Pick a held-out applicant (never seen during training) and score them with the trained LightGBM model.

Sample applicant

#316484 income 279,000 credit 479,700 age 66

Applicant #316484

17.1%

predicted default probability

Low

Design decisions

Business Rules

Credit Risk Intelligence

Home Credit Default Risk

Data Overview

Risk Prediction

Explainability

Business Rules

Ask the Data

Business Rules

Plain IF-THEN rules extracted from the model, each validated against real, held-out outcomes, not just how clean they read.

8.07%

Baseline default rate

0.393

Surrogate fidelity (vs full model)

3

Rules that passed quality filters

RULE 1

Low

IF years employed is above 4.52 AND loan term (years) is at most 0.90 AND loan term (years) is above 0.81

3.36% (2,068 applicants)

Coverage

1.93%

Actual default rate

8.07%

Baseline

Narrow range on 'loan term (years)' (0.81 to 0.90), bounded from both sides by this single decision tree. Treat as a lower-confidence pattern worth re-checking on more data before codifying into policy, unlike the other rules here which use one wide threshold per feature.

RULE 2

High

IF years employed is at most 4.52 AND region risk rating (1 best, 3 worst) is above 1.50 AND loan term (years) is at most 1.79

24.09% (14,816 applicants)

Coverage

12.93%

Actual default rate

8.07%

Baseline

RULE 3

Low

IF years employed is above 4.52 AND loan term (years) is above 0.90 AND number of active bureau credits is at most 1.50

20.05% (12,334 applicants)

Coverage

5.21%

Actual default rate

8.07%

Baseline

Live at credit-risk-platform.vercel.app

13

Ask the Data, a live example

The screenshot shows a web application titled 'Credit Risk Intelligence' with a sidebar menu containing 'Data Overview', 'Risk Prediction', 'Explainability', 'Business Rules', and 'Ask the Data'. The main content area is titled 'Ask the Data' and includes a dropdown menu 'What data can I ask about?'. A question is entered in a dark box: 'What is the default rate for applicants with a refused prior application versus those without?'. The answer is displayed in a light box, stating that applicants with a refused prior application have a higher default rate (11.29%) compared to those without (7.56%). Below the answer is a link 'View generated SQL'. There are also 'Example questions' listed in a row of buttons, and a text input field with a 'Send' button at the bottom.

Credit Risk Intelligence
Home Credit Default Risk

- Data Overview
- Risk Prediction
- Explainability
- Business Rules
- Ask the Data

Ask the Data

Ask a plain-English question, it becomes a validated SQL query behind the scenes. Follow-up questions keep context from earlier in this conversation.

What data can I ask about?

What is the default rate for applicants with a refused prior application versus those without?

Applicants who had a prior application refused have a default rate of about 11.29%, which is higher than the 7.56% default rate for those without a refused prior application. This suggests that having a refused application in the past is associated with a greater likelihood of default. The sample sizes are quite large, with over 42,000 applicants in the refused group and over 265,000 in the non-refused group, so these results are reliable.

► View generated SQL

Example questions

How many applicants took cash loans versus revolving loans? What's the average income by education type?

What is the default rate for applicants with a refused prior application versus those without? How many applicants have more than 2 active bureau credits?

Ask a question about the applicants...

Send

Design decisions

Two deployment paths, one codebase

Production (live links)

- Frontend: React SPA on **Vercel**
- Backend: FastAPI on **Render**
- Database: **Neon Postgres**, capped at 400k rows on the two largest tables to stay under the free tier's 512MB limit
- Keep-alive: GitHub Actions cron hitting `/health` every 10 minutes against Render cold starts

Local / Docker

- `docker-compose up --build`, backend + frontend containers
- Zero dataset setup required: a committed ~4MB demo database powers the chatbot and prediction dropdown out of the box
- 3-tier fallback in `src/data/db.py`: `DATABASE_URL` → full local DB if built → committed demo DB
- Verified empirically: stripped the env var and the full DB, confirmed chat answers matched the demo dataset's exact row counts

Known limitations and future improvements

Limitations

- PR-AUC (0.263) is modest in absolute terms, which follows from the 8% base rate rather than reflecting a weakness in the model
- Feature scope currently covers application data plus bureau aggregates; payment-history tables (POS_CASH, credit_card, installments) are not yet aggregated
- Surrogate rules trade fidelity (0.393) for actionability, as a deliberate design choice
- Free-tier hosting introduces cold starts and row caps on the production database
- No authentication on any endpoint, which is suitable for this demo but would need to be added for production use

Given more time

- Aggregate the remaining Home Credit tables for a likely ROC-AUC gain
- k-fold cross-validation and probability calibration
- A monotonic-constrained surrogate tree for better rule fidelity without losing actionability
- Per-user rate limiting and basic auth in front of the chatbot
- A model/data drift monitoring job

Thank you



Live app: credit-risk-platform.vercel.app

API: credit-risk-api-l0nw.onrender.com

Full source, README, and this deck: github.com/meetvirani8833/credit-risk-platform